



Join us at [javatechcommunity.com](https://javatechcommunity.com) for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely **FREE!**

## JAVA INTERVIEW - CODING ROUND



### TOP-10

# JAVA 8 STREAMS API PROGRAMS



**Copyright Notice:**

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - [support@javatechcommunity.com](mailto:support@javatechcommunity.com)



Join us at [javatechcommunity.com](https://javatechcommunity.com) for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely **FREE!**

## 01. USING JAVA 8, RETURN A LIST CONTAINING ONLY EVEN NUMBERS FROM A LIST OF INTEGERS.

```
// Way 1: Using Streams (Java 8) to filter even numbers from a list of integers
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class EvenNumbersWay1 {
    public static void main(String[] args) {
        // Input list of integers
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);

        // Filter even numbers using Streams
        List<Integer> evenNumbers = numbers.stream()
            .filter(n -> n % 2 == 0) // Condition for even numbers
            .collect(Collectors.toList()); // Collect the result

        // Output the result
        System.out.println("Even Numbers: " + evenNumbers);
    }
}
```

```
// Way 2: Using a Basic Loop to filter even numbers from a list of integers
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class EvenNumbersWay2 {
    public static void main(String[] args) {
        // Input list of integers
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);

        // Filter even numbers using a basic loop
        List<Integer> evenNumbers = new ArrayList<>();
        for (Integer num : numbers) {
            if (num % 2 == 0) {
                evenNumbers.add(num);
            }
        }

        // Output the result
        System.out.println("Even Numbers: " + evenNumbers);
    }
}
```



### Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com



Join us at [javatechcommunity.com](https://javatechcommunity.com) for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely **FREE!**

## 02. USING JAVA 8, FIND THE MAXIMUM VALUE IN A LIST OF INTEGERS.

```
// Way 1: Using Streams (Java 8) to find the maximum value in a list of integers
import java.util.Arrays;
import java.util.List;
import java.util.Optional;

public class MaxValueWay1 {
    public static void main(String[] args) {
        // Input list of integers
        List<Integer> numbers = Arrays.asList(1, 3, 7, 2, 9, 5);

        // Find maximum value using Streams
        Optional<Integer> max = numbers.stream()
            .max(Integer::compare);

        // Output the result
        System.out.println("Maximum Value: " + max.orElse(null));
    }
}
```

```
// Way 2: Using a Basic Loop to find the maximum value in a list of integers
import java.util.Arrays;
import java.util.List;

public class MaxValueWay2 {
    public static void main(String[] args) {
        // Input list of integers
        List<Integer> numbers = Arrays.asList(1, 3, 7, 2, 9, 5);

        // Find maximum value using a basic loop
        int max = Integer.MIN_VALUE;
        for (Integer num : numbers) {
            if (num > max) {
                max = num;
            }
        }

        // Output the result
        System.out.println("Maximum Value: " + max);
    }
}
```



### Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com



Join us at [javatechcommunity.com](https://javatechcommunity.com) for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely **FREE!**

### 03. USING JAVA 8, SORT A LIST OF INTEGERS IN ASCENDING ORDER.

```
// Way 1: Using Streams (Java 8) to sort a list of integers in ascending order
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class SortListWay1 {
    public static void main(String[] args) {
        // Input list of integers
        List<Integer> numbers = Arrays.asList(5, 1, 3, 7, 2);

        // Sort the list using Streams
        List<Integer> sortedNumbers = numbers.stream()
            .sorted() // Sort in natural order
            .collect(Collectors.toList());

        // Output the result
        System.out.println("Sorted List: " + sortedNumbers);
    }
}
```

```
// Way 2: Using Collections.sort() to sort a list of integers in ascending order
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

public class SortListWay2 {
    public static void main(String[] args) {
        // Input list of integers
        List<Integer> numbers = Arrays.asList(5, 1, 3, 7, 2);

        // Sort the list using Collections.sort()
        Collections.sort(numbers);

        // Output the result
        System.out.println("Sorted List: " + numbers);
    }
}
```



**Copyright Notice:**

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com





Join us at [javatechcommunity.com](https://javatechcommunity.com) for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely **FREE!**

#### 4. USING JAVA 8, COUNT THE ELEMENTS IN A LIST THAT ARE GREATER THAN 5.

```
// Way 1: Using Streams (Java 8) to count elements greater than 5
import java.util.Arrays;
import java.util.List;
```

```
public class CountGreaterThanFiveWay1 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 3, 7, 2, 9, 5);
        long count = numbers.stream()
            .filter(n -> n > 5)
            .count();
        System.out.println("Input: " + numbers);
        System.out.println("Count of Numbers > 5: " + count);
    }
}
```

```
// Way 2: Using a Basic Loop to count elements greater than 5
import java.util.Arrays;
import java.util.List;
```

```
public class CountGreaterThanFiveWay2 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 3, 7, 2, 9, 5);
        int count = 0;
        for (Integer num : numbers) {
            if (num > 5) {
                count++;
            }
        }
        System.out.println("Input: " + numbers);
        System.out.println("Count of Numbers > 5: " + count);
    }
}
```



**Copyright Notice:**

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - [support@javatechcommunity.com](mailto:support@javatechcommunity.com)



Join us at [javatechcommunity.com](https://javatechcommunity.com) for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely **FREE!**

## 5. USING JAVA 8, RETRIEVE ALL DISTINCT ELEMENTS FROM A LIST.

```
// Way 1: Using Streams (Java 8) to retrieve distinct elements from a list
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class DistinctElementsWay1 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 2, 3, 4, 4, 5);
        List<Integer> distinctNumbers = numbers.stream()
                                                .distinct()
                                                .collect(Collectors.toList());

        System.out.println("Input: " + numbers);
        System.out.println("Distinct Elements: " + distinctNumbers);
    }
}
```

```
// Way 2: Using a Basic Loop to retrieve distinct elements from a list
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class DistinctElementsWay2 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 2, 3, 4, 4, 5);
        List<Integer> distinctNumbers = new ArrayList<>();
        for (Integer num : numbers) {
            if (!distinctNumbers.contains(num)) {
                distinctNumbers.add(num);
            }
        }
        System.out.println("Input: " + numbers);
        System.out.println("Distinct Elements: " + distinctNumbers);
    }
}
```



### Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com



Join us at [javatechcommunity.com](https://javatechcommunity.com) for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely **FREE!**

## 6. USING JAVA 8, SKIP THE FIRST 2 ELEMENTS OF A LIST AND RETURN THE REST.

```
// Way 1: Using Streams (Java 8) to skip the first 2 elements
```

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class SkipElementsWay1 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
        List<Integer> skippedNumbers = numbers.stream()
            .skip(2) // Skip first 2 elements
            .collect(Collectors.toList());

        System.out.println("Input: " + numbers);
        System.out.println("After Skipping First 2 Elements: " + skippedNumbers);
    }
}
```

```
// Way 2: Using a Basic Loop to skip the first 2 elements
```

```
import java.util.Arrays;
import java.util.List;

public class SkipElementsWay2 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
        List<Integer> skippedNumbers = numbers.subList(2, numbers.size()); // Skip first 2
        System.out.println("Input: " + numbers);
        System.out.println("After Skipping First 2 Elements: " + skippedNumbers);
    }
}
```



### Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com



Join us at [javatechcommunity.com](https://javatechcommunity.com) for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely **FREE!**

## 7. USING JAVA 8, CONVERT A LIST OF INTEGERS TO A SET TO REMOVE DUPLICATES.

```
// Way 1: Using Streams (Java 8) to remove duplicates
import java.util.Arrays;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

public class RemoveDuplicatesWay1 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 2, 3, 4, 4, 5);
        Set<Integer> uniqueNumbers = numbers.stream()
                                            .collect(Collectors.toSet());

        System.out.println("Input: " + numbers);
        System.out.println("Unique Numbers: " + uniqueNumbers);
    }
}

// Way 2: Using a Basic Loop to remove duplicates
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class RemoveDuplicatesWay2 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 2, 3, 4, 4, 5);
        Set<Integer> uniqueNumbers = new HashSet<>(numbers); // HashSet removes duplicates
        System.out.println("Input: " + numbers);
        System.out.println("Unique Numbers: " + uniqueNumbers);
    }
}
```



### Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com



Join us at [javatechcommunity.com](https://javatechcommunity.com) for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely **FREE!**

## 8. USING JAVA 8, GROUP ELEMENTS BY A SPECIFIC PROPERTY, SUCH AS AGE.

```
class Person {
    String name;
    int age;

    // Constructor
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Getter for age
    public int getAge() {
        return age;
    }

    @Override
    public String toString() {
        return name + " (" + age + ")";
    }
}

// Solution using Java 8 Streams to group elements by age
import java.util.*;
import java.util.stream.Collectors;

public class GroupByPropertyStream {
    public static void main(String[] args) {
        // List of people with Indian names
        List<Person> people = Arrays.asList(
            new Person("Dhoni", 41),
            new Person("Sachin", 50),
            new Person("Kohli", 34),
            new Person("Rohit", 34),
            new Person("Pant", 25)
        );

        // Group people by age using Streams
        Map<Integer, List<Person>> groupedByAge = people.stream()
            .collect(Collectors.groupingBy(Person::getAge));

        // Print the grouped result
        System.out.println("Grouped By Age (Java 8): " + groupedByAge);
    }
}

// Solution using a Basic Loop to group elements by age
import java.util.*;

public class GroupByPropertyLoop {
    public static void main(String[] args) {
        // List of people with Indian names
        List<Person> people = Arrays.asList(
            new Person("Dhoni", 41),
            new Person("Sachin", 50),
            new Person("Kohli", 34),
            new Person("Rohit", 34),
            new Person("Pant", 25)
        );

        // Group people by age using a basic loop
        Map<Integer, List<Person>> groupedByAge = new HashMap<>();
        for (Person person : people) {
            groupedByAge.putIfAbsent(person.getAge(), new ArrayList<>());
            groupedByAge.get(person.getAge()).add(person);
        }

        // Print the grouped result
        System.out.println("Grouped By Age (Basic Loop): " + groupedByAge);
    }
}
```



### Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - [support@javatechcommunity.com](mailto:support@javatechcommunity.com)





Join us at [javatechcommunity.com](https://javatechcommunity.com) for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely **FREE!**

## 9. USING JAVA 8, REDUCE A LIST OF INTEGERS TO THEIR SUM.

```
// Way 1: Using Streams (Java 8) to calculate the sum of integers
import java.util.Arrays;
import java.util.List;

public class SumOfListWay1 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        int sum = numbers.stream()
            .reduce(0, Integer::sum);
        System.out.println("Input: " + numbers);
        System.out.println("Sum: " + sum);
    }
}
```

```
// Way 2: Using a Basic Loop to calculate the sum of integers
import java.util.Arrays;
import java.util.List;

public class SumOfListWay2 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        int sum = 0;
        for (Integer num : numbers) {
            sum += num;
        }
        System.out.println("Input: " + numbers);
        System.out.println("Sum: " + sum);
    }
}
```



### Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - [support@javatechcommunity.com](mailto:support@javatechcommunity.com)





Join us at [javatechcommunity.com](https://javatechcommunity.com) for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely **FREE!**

## 10. USING JAVA 8, CONVERT ALL STRINGS IN A LIST TO UPPERCASE.

```
// Way 1: Using Streams (Java 8) to convert strings to uppercase
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class ConvertToUpperCaseWay1 {
    public static void main(String[] args) {
        List<String> strings = Arrays.asList("java", "stream", "api");
        List<String> upperCaseStrings = strings.stream()
                                                .map(String::toUpperCase)
                                                .collect(Collectors.toList());

        System.out.println("Input: " + strings);
        System.out.println("Uppercase Strings: " + upperCaseStrings);
    }
}
```

```
// Way 2: Using a Basic Loop to convert strings to uppercase
import java.util.Arrays;
import java.util.ArrayList;
import java.util.List;

public class ConvertToUpperCaseWay2 {
    public static void main(String[] args) {
        List<String> strings = Arrays.asList("java", "stream", "api");
        List<String> upperCaseStrings = new ArrayList<>();
        for (String str : strings) {
            upperCaseStrings.add(str.toUpperCase());
        }
        System.out.println("Input: " + strings);
        System.out.println("Uppercase Strings: " + upperCaseStrings);
    }
}
```



### Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - [support@javatechcommunity.com](mailto:support@javatechcommunity.com)

# DO YOU WANT ACCESS TO ALL THESE Q&A IN ONE PLACE? ✨📖

Java Tech Community

x

←

→

🔄

🔍 javatechcommunity.com

Download Exclusive Premium ePDF Resources in the Java Tech Community Bundle – Curated by a Top Indian Java Community Expert!



HTML, CSS & JS

Latest interview questions on web fundamentals.



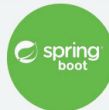
Angular & React

Important interview questions for modern web frameworks.



Java & J2EE

Latest key interview questions for enterprise Java roles.



Spring Boot

Important interview questions for Spring Boot development.



Hibernate

Latest crucial interview questions for Hibernate ORM.



REST API & Microservices

Important interview questions for APIs and microservices.



CI/CD & DevOps

Latest interview questions for DevOps practices.



AWS

Important interview questions for cloud-based Java applications.



Data structures and algorithms

Latest interview questions on data structures for Java.



Agile Framework

Adopt Agile practices for collaborative development. Improve productivity in Java-based projects.



Coding Round Essentials

Latest key coding interview questions for Java developers.



HR & Manager Interview Essentials

Important questions for HR and managerial interviews.

## COMMUNITY WEBSITE LAUNCH 🚀

🌐 ACCESS THE COMPLETE NOTES ON THE JAVA TECH COMMUNITY WEBSITE.

 **LAUNCH DATE: DECEMBER 22ND, 2024 (BETA VERSION)**  
👉 DON'T FORGET TO MARK YOUR CALENDAR!

## STAY UPDATED

📱 UPDATES WILL BE SHARED IN THIS COMMUNITY CHANNEL AND ON OUR INSTAGRAM CHANNEL.

**FOLLOW US ON INSTAGRAM** @JAVA\_TECH\_COMMUNITY FOR REAL-TIME UPDATES!

**Copyright Notice:**  
This document is strictly for personal use only and not for redistribution or commercial purposes.  
If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com